

Relatório Técnico – Trabalho Final

Princípios e Padrões de Projetos

Objetivo

Este trabalho consistiu no desenvolvimento de um framework orientado a objetos para aplicações do tipo Quiz, construído de forma genérica o suficiente para permitir que diferentes aplicações fossem criadas sobre ele sem alterar seu código-fonte.

Arquitetura

- **Framework (quiz.framework)** – gerenciamento das perguntas, fluxo de execução do quiz, cálculo da pontuação, regras de negócio e abstração da interface de exibição.
- **Aplicações Cliente (quiz.aplicacao)** – definem as perguntas de cada quiz, configuram a execução, escolhem a estratégia de pontuação e implementam a interface com o usuário.

Essa divisão permite que o mesmo framework seja reaproveitado por diferentes tipos de quiz, sem duplicação de código.

Padrões de Projeto Utilizados

- **Template Method** – a classe abstrata *AbstractQuiz* define, no método final *iniciar()*, a sequência fixa de configuração e disparo de um quiz (criar interface de exibição, montar perguntas, capturar jogadores através da própria interface, definir a estratégia de pontuação, registrar observadores e executar o motor do quiz). Os passos variáveis (quais perguntas, qual pontuação, qual interface) são implementados pelas subclasses concretas de cada aplicação (*QuizProgramacao* e *QuizConhecimentosGerais*).
- **Strategy** – encapsula diferentes políticas de pontuação. O Quiz de Programação usa a estratégia padrão (pontos fixos por acerto); o Quiz de Conhecimentos Gerais usa a estratégia baseada em tempo de resposta, que também é exibida ao jogador a cada pergunta e no resultado final. Uma terceira estratégia, com penalização por erro, também está disponível no framework.
- **Factory** – constrói as perguntas a partir das anotações definidas em cada aplicação, reduzindo o acoplamento entre o núcleo do framework e as classes concretas.
- **Observer** (implementação opcional) – desacopla o *QuizEngine* da forma como o progresso da partida é exibido (Placar), permitindo trocar ou adicionar formas de visualização sem alterar o núcleo do framework. A multiplicidade de jogadores é suportada nativamente pelo *QuizEngine* (lista de *Jogador*), independentemente do número de observadores registrados — dado que a aplicação roda em processo único, sem paralelismo real entre jogadores, um único Placar compartilhado é suficiente para agregar e exibir o progresso de todos simultaneamente.

Como implementação opcional, também foi utilizado um mecanismo baseado em **Annotations**, que permite que as perguntas sejam descobertas e registradas automaticamente pelo framework, sem cadastro manual.

Bibliotecas Externas Utilizadas

Para implementar a descoberta automática de classes anotadas com `@QuizPergunta` dentro de um pacote, foi necessário incluir a biblioteca **Reflections** (v0.10.2), já que a API padrão de reflection do Java não oferece um mecanismo nativo para listar todas as classes de um pacote — ela só permite inspecionar uma classe já conhecida em tempo de execução. A biblioteca **Javassist** (v3.28.0-GA) foi incluída como dependência exigida pelo **Reflections** para a varredura de bytecode no classpath. Como dependência transitiva do próprio **Reflections** (que a utiliza internamente para registrar mensagens de log), também foi necessário incluir o **SLF4J API** (v1.7.36); sem ele, a aplicação falha em tempo de execução com *NoClassDefFoundError* ao instanciar o **Reflections**. O **SLF4J NOP** (v1.7.36) foi incluído como provedor de log "no-op", apenas para satisfazer a exigência do SLF4J por uma implementação registrada, sem gerar nenhuma saída de log no console. Todas as bibliotecas foram adicionadas manualmente ao projeto (pasta *lib/* na raiz, configuradas como *library* no IntelliJ), sem uso de gerenciador de dependências (Maven/Gradle). Elas atuam apenas como suporte técnico ao mecanismo de descoberta via reflection — nenhum framework de quiz pronto foi utilizado; toda a lógica de negócio, fluxo do jogo, pontuação e padrões de projeto (Template Method, Strategy, Factory, Observer) foram implementados integralmente pelo grupo.

Aplicações Desenvolvidas

- **Quiz de Programação**, com interface gráfica em Java Swing e pontuação padrão;
- **Quiz de Conhecimentos Gerais**, com interface em modo console e pontuação baseada em tempo de resposta.

Ambas utilizam exatamente o mesmo núcleo de execução do framework, diferenciando-se apenas nas perguntas cadastradas, na estratégia de pontuação e na forma de interação com o usuário.

Considerações Finais

O desenvolvimento permitiu aplicar na prática os princípios de orientação a objetos e os padrões de projeto estudados na disciplina, favorecendo a reutilização de código, a extensibilidade do framework e a redução do acoplamento entre os componentes do sistema.